Package 'lpmec'

June 10, 2026

Title Measurement Error Analysis and Correction Under Identification Restrictions

Version 1.1.4

Author Connor Jerzak [aut, cre], Stephen Jessee [aut]

Description Implements methods for analyzing latent variable models with measurement error correction, including Item Response Theory (IRT) models. Provides tools for various correction methods such as Bayesian Markov Chain Monte Carlo (MCMC), over-imputation, bootstrapping for robust standard errors, Ordinary Least Squares (OLS), and Instrumental Variables (IV) based approaches. Supports flexible specification of observable indicators and groupings for latent variable analyses in social sciences and other fields. Methods are described in a working paper (2025) <doi:10.48550/arXiv.2507.22218>.

Depends R (>= 3.5.0)

License GPL-3

Encoding UTF-8

LazyData false

Maintainer Connor Jerzak <connor.jerzak@gmail.com>

Imports reticulate,
stats,
sensemakr,
pscl,
AER,
sandwich,
mvtnorm,
Amelia,
emIRT,
gtools

Suggests testthat (>= 3.0.0),
knitr,
rmarkdown

SystemRequirements Python (>= 3.10) with jax, numpy, numpyro (optional; for NumPyro backend via reticulate)

VignetteBuilder knitr

Config/testthat/edition 3

RoxygenNote 7.3.3

URL https://github.com/cjerzak/lpmec-software

BugReports https://github.com/cjerzak/lpmec-software/issues

Contents

`build_backend` *A function to build the environment for lpmec. Builds a conda environment in which 'JAX', 'numpyro', and 'numpy' are installed. Users can also create a conda environment where 'JAX' and 'numpy' are installed themselves.*

Description

A function to build the environment for lpmec. Builds a conda environment in which 'JAX', 'numpyro', and 'numpy' are installed. Users can also create a conda environment where 'JAX' and 'numpy' are installed themselves.

Usage

```
build_backend(conda_env = "lpmec", conda = "auto")
```

Arguments

`conda_env`	(default = `"lpmec"`) Name of the conda environment in which to place the backends.
`conda`	(default = `auto`) The path to a conda executable. Using `"auto"` allows reticulate to attempt to automatically find an appropriate conda binary.

Value

Invisibly returns NULL; this function is used for its side effects of creating and configuring a conda environment for `lpmec`. This function requires an Internet connection. You can find out a list of conda Python paths via: `Sys.which("python")`

Examples

```
## Not run:
# Create a conda environment named "lpmec"
# and install the required Python packages (jax, numpy, etc.)
build_backend(conda_env = "lpmec", conda = "auto")

# If you want to specify a particular conda path:
# build_backend(conda_env = "lpmec", conda = "/usr/local/bin/conda")

## End(Not run)
```

`infer_orientation_signs` *Infer orientation signs for each observable indicator*

Description

This helper analyzes observable indicators and returns a numeric vector of `1` or `-1` for use with the `orientation_signs` argument in `lpmec`. Each sign is chosen so that the correlation between the oriented indicator and either the outcome `Y` or the first principal component of the indicators is positive.

Usage

```
infer_orientation_signs(Y, observables, method = c("Y", "PC1"))
```

Arguments

`Y`	Numeric outcome vector. Only used when `method = "Y"`.
`observables`	A matrix or data frame of binary observable indicators.
`method`	Character string specifying how to orient the indicators.
	`"Y"` orient each indicator so that its correlation with `Y` is positive.
	`"PC1"` orient each indicator so that its correlation with the first principal component of `observables` is positive.
	Default is `"Y"`.

Value

A numeric vector of length `ncol(observables)` containing `1` or `-1`.

Examples

```
set.seed(1)
Y <- rnorm(10)
obs <- data.frame(matrix(sample(c(0,1), 20, replace = TRUE), ncol = 2))
infer_orientation_signs(Y, obs)
```

`KnowledgeVoteDuty` *KnowledgeVoteDuty: Survey Respondents' Views of Voting as a Duty and Political Knowledge Questions*

Description

KnowledgeVoteDuty is a modified set of responses to a small set of questions on the American National Election Study's 2024 Time Series Study. These data only include respondents who had non-missing values on all of the variables included, dropping respondents with one or more missing values.

Usage

```
data(KnowledgeVoteDuty)
```

Format

A data frame with 3,059 observations and 5 variables:

voteduty Whether respondents feel that voting is a duty or a choice. Values range from 1 to 7, with 1 being "Very strongly a duty" and 7 being "Very strongly a choice," created based on variable V241218x.

SenateTerm Dummy variable (0 or 1) for whether respondent correctly stated the length of a U.S. Senate term. Created based on variable V241612.

SpendLeast Dummy variable (0 or 1) for whether respondent correctly identified "Foreign aid" from a list as the category the federal government spends the least on. Created based on variable V241613.

HouseParty Dummy variable (0 or 1) for whether respondent correctly identified the political party that currently has the most members in the U.S. House of Representatives. Created based on variable V241614.

SenateParty Dummy variable (0 or 1) for whether respondent correctly identified the political party that currently has the most members in the U.S. Senate. Created based on variable V241615.

References

American National Election Studies. 2024. ANES 2024 Time Series Study Full Release [dataset and documentation]. Available at electionstudies.org.

Examples

```
data(KnowledgeVoteDuty)
voteduty <- KnowledgeVoteDuty$voteduty
knowledge <- scale(rowMeans(KnowledgeVoteDuty[ , -1]))
summary(lm(voteduty ~ knowledge))
```

lpmec *lpmec*

Description

Implements latent variable models with measurement error correction

Usage

```
lpmec(
  Y,
  observables,
  observables_groupings = colnames(observables),
  orientation_signs = NULL,
  make_observables_groupings = FALSE,
  n_boot = 32L,
  n_partition = 10L,
  partition_aggregation = "median",
  partition_aggregation_probs = c(0.01, 0.99),
  boot_basis = 1:length(Y),
  bootstrap_method = c("n_out_of_n", "m_out_of_n", "subsampling", "auto"),
  boot_m = NULL,
  boot_m_rule = c("power", "fixed", "grid_stability"),
  boot_m_exponent = 0.7,
  boot_m_grid = NULL,
  boot_m_replace = NULL,
  boot_ci_type = c("auto", "root", "percentile"),
  boot_alpha = 0.05,
  boot_rate = c("sqrt_n", "custom"),
  boot_tau = NULL,
  partition_set = NULL,
  fix_partitions = TRUE,
  seed = NULL,
  return_intermediaries = TRUE,
  ordinal = FALSE,
  estimation_method = "em",
  latent_estimation_fn = NULL,
 mcmc_control = list(backend = "pscl", n_samples_warmup = 500L, n_samples_mcmc = 1000L,
    batch_size = 512L, chain_method = "parallel", subsample_method = "full",
    anchor_parameter_id = NULL, n_thin_by = 1L, n_chains = 2L, outcome_prior =
    list(calibration = "data"), joint2_prior = list()),
  conda_env = "lpmec",
  conda_env_required = FALSE
)
```

Arguments

Y	A vector of observed outcome variables
observables	A matrix of observable indicators used to estimate the latent variable

observables_groupings
: A vector specifying groupings for the observable indicators. Default is column names of observables.

orientation_signs
: (optional) A numeric vector of length equal to the number of columns in 'observables', containing 1 or -1 to indicate the desired orientation of each column. If provided, each column of 'observables' will be oriented by this sign before analysis. Default is NULL (no orientation applied).

make_observables_groupings
: Logical. If TRUE, creates dummy variables for each level of the observable indicators. Default is FALSE.

n_boot
: Non-negative integer. Number of bootstrap iterations. Use `0` to disable bootstrap resampling and fit only the original sample. Default is 32.

n_partition
: Positive integer. Number of split-half partitions for each bootstrap iteration. When `n_boot = 0`, this still controls how many original-sample partition runs are aggregated. Default is 10.

partition_aggregation
: Aggregation strategy for combining estimates across partitions within each bootstrap iteration. Default is `"median"`. Options are `"median"`, `"winsorized_mean"`, `"trimmed_mean"`, or a custom function that accepts a numeric vector and returns one numeric value.

partition_aggregation_probs
: Numeric vector of length 2 used by `"winsorized_mean"` and `"trimmed_mean"`. For winsorization, values are clipped to these quantiles before averaging. For trimming, values outside these quantiles are dropped before averaging. Default is `c(0.01, 0.99)`.

boot_basis
: Vector of indices or grouping variable for stratified bootstrap. Default is 1:length(Y).

bootstrap_method
: Resampling method for uncertainty. Options are `"n_out_of_n"`, `"m_out_of_n"`, `"subsampling"`, and `"auto"`. The default `"n_out_of_n"` preserves historical row bootstrap behavior. Use `"subsampling"` or `"m_out_of_n"` with `boot_ci_type = "root"` for the formal nonsmooth median route.

boot_m
: Optional exact m for m-out-of-n bootstrap or subsampling.

boot_m_rule
: Rule used when `boot_m = NULL`. `"power"` uses `floor(n^boot_m_exponent)`; `"fixed"` requires `boot_m`; `"grid_stability"` records a candidate grid and currently selects the grid value nearest the power-rule value.

boot_m_exponent
: Exponent used by `boot_m_rule = "power"`. Default is `0.70`.

boot_m_grid
: Optional candidate grid for m-sensitivity diagnostics.

boot_m_replace
: Optional logical override for row replacement in m-out-of-n/subsampling. By default, replacement is used for `"n_out_of_n"` and `"m_out_of_n"`, and not used for `"subsampling"`.

boot_ci_type
: Confidence interval type. `"auto"` uses percentile intervals for `"n_out_of_n"` and root-scaled intervals for m < n.

boot_alpha
: Confidence interval tail probability. Default is 0.05.

boot_rate
: Rate used by root-scaled intervals. The current formal path uses `"sqrt_n"`; `"custom"` requires `boot_tau`.

boot_tau
: Optional function used when `boot_rate = "custom"`.

partition_set Optional user-supplied fixed partition list. Each element must contain split1_names, split2_names, and optionally partition_id.

fix_partitions Logical. If TRUE, draw or accept the finite partition set once and hold it fixed across original and resampled fits.

seed Optional seed used for reproducible partitions and resamples.

return_intermediaries
Logical. If TRUE, returns intermediate results. Default is TRUE.

ordinal Logical indicating whether the observable indicators are ordinal (TRUE) or binary (FALSE).

estimation_method
Character specifying the estimation approach. Options include:

- "em" (default): Uses expectation-maximization via emIRT package. Supports both binary (via emIRT::binIRT) and ordinal (via emIRT::ordIRT) indicators.
- "pca": First principal component of observables.
- "averaging": Uses feature averaging.
- "mcmc": Markov Chain Monte Carlo estimation using either pscl::ideal (R backend) or numpyro (Python backend)
- "mcmc_joint": Joint Bayesian model that simultaneously estimates latent variables and outcome relationship using numpyro
- "mcmc_joint2": NumPyro mixed factor-analysis benchmark with binary indicators and continuous Y in one factor model
- "mcmc_overimputation": Two-stage MCMC approach with measurement error correction via over-imputation
- "custom": In this case, latent estimation performed using latent_estimation_fn.

latent_estimation_fn
Custom function for estimating latent trait from observables if estimation_method="custom" (optional). The function should accept a matrix of observables (rows are observations) and return a numeric vector of length equal to the number of observations.

mcmc_control A list indicating parameter specifications if MCMC used.

backend Character string indicating the MCMC engine to use. Valid options are "pscl" (default, uses the R-based pscl::ideal function) or "numpyro" (uses the Python numpyro package via reticulate).

n_samples_warmup Integer specifying the number of warm-up (burn-in) iterations before samples are collected. Default is 500.

n_samples_mcmc Integer specifying the number of post-warmup MCMC iterations to retain. Default is 1000.

batch_size Integer row subsample size used when subsample_method = "batch" with the NumPyro backend. Must be between 1 and nrow(observables) - 1. Default is 512.

subsample_method Character string for NumPyro likelihood evaluation. Use "full" (default) for all rows or "batch" for experimental HMCECS row subsampling with batch_size.

chain_method Character string passed to numpyro specifying how to run multiple chains. Options: "parallel" (default), "sequential", or "vectorized".

anchor_parameter_id Optional 1-based observable-column index used by NumPyro MCMC backends to anchor item difficulty orientation. If omitted or NULL, automatic orientation is used.

n_thin_by Integer indicating the thinning factor for MCMC samples. Default is 1.

n_chains Integer specifying the number of parallel MCMC chains to run. Default is 2.

outcome_prior List controlling "mcmc_joint" outcome-model priors for the NumPyro backend. By default, calibration = "data" centers the intercept prior at mean(Y) and scales intercept, slope, and residual-sigma priors by sd(Y). Use calibration = "legacy" to restore the previous unit-scale priors, or provide numeric overrides for intercept_mean, intercept_sd, slope_mean, slope_sd, and sigma_sd. The optional scale_floor sets the minimum scale used for data-calibrated priors.

joint2_prior List controlling "mcmc_joint2" priors. Defaults are lambda_mean = 0, lambda_sd = 2, psi_shape = 0.0005, and psi_scale = 0.0005. For item loadings, lambda_mean and lambda_sd parameterize the raw loading scale before the positive softplus transform; all item loadings are positive by default.

conda_env A character string specifying the name of the conda environment to use via reticulate. Default is "lpmec".

conda_env_required
A logical indicating whether the specified conda environment must be strictly used. If TRUE, an error is thrown if the environment is not found. Default is FALSE.

Details

This function implements a latent variable analysis with measurement error correction. It fits the original sample and, when n_boot >= 1, performs bootstrap resampling for uncertainty estimates. Each original or bootstrap sample is analyzed with one or more split-half partitions. For each partition, it calls the lpmec_onerun function to estimate latent variables and apply various correction methods. The results are then aggregated across partitions and bootstrap iterations to produce final estimates and, when bootstrap draws are available, bootstrap standard errors.

For partition_aggregation = "median", the finite-partition median is a nonsmooth aggregation rule. The ordinary n-out-of-n row bootstrap remains available through bootstrap_method = "n_out_of_n" for backward compatibility. The formal nonsmooth-functional route is bootstrap_method = "subsampling" or "m_out_of_n" with boot_ci_type = "root". In that route, the same realized partition set is held fixed across the original sample and all resamples, each resample reruns the full latent-score and correction pipeline, and confidence intervals invert the empirical distribution of sqrt(m) * (theta_boot - theta_hat) at the original sqrt(n) rate.

Value

A list containing various estimates and statistics (in snake_case):

- Naive, IV, corrected IV, and corrected OLS estimates: ols_*, iv_*, corrected_iv_*, and corrected_ols_*. Bootstrap uncertainty summaries use suffixes _se, _lower, _upper, and _tstat where applicable.
- var_est_split and var_est_split_se: Aggregated split-half measurement-error variance and, when bootstrap draws are available, its bootstrap standard error.
- bayesian_ols_*_outer_normed and bayesian_ols_*_inner_normed: MCMC coefficient summaries. The *_parametric standard-error fields retain within-run posterior uncertainty, while the non-parametric standard-error and interval fields summarize bootstrap variation.

- `m_stage_1_erv*` and `m_reduced_erv*`: Extreme robustness values and bootstrap uncertainty summaries for the first-stage and reduced-form regressions.
- `mcmc_joint2_*`: NumPyro `"mcmc_joint2"` diagnostics, including effective-sample-size percentages, maximum R-hat, divergent transitions, mean accept probability, and orientation diagnostics.
- `x_est1` and `x_est2`: Split-half latent variable estimates from the original sample.
- `Intermediary_*`: Per-run original-sample and bootstrap outputs, returned only when `return_intermediaries = TRUE`.

References

Jerzak, C. T. and Jessee, S. A. (2025). Attenuation Bias with Latent Predictors. arXiv:2507.22218 [stat.AP]. https://arxiv.org/abs/2507.22218

Examples

```
# Generate some example data
set.seed(123)
Y <- rnorm(1000)
observables <- as.data.frame(matrix(sample(c(0,1), 1000*10, replace = TRUE), ncol = 10))

# Run the bootstrapped analysis
results <- lpmec(Y = Y,
                 observables = observables,
                 n_boot = 10,    # small values for illustration only
                 n_partition = 5 # small for size
                 )

# Use a winsorized mean across partitions
results_winsorized <- lpmec(Y = Y,
                            observables = observables,
                            n_boot = 10,
                            n_partition = 5,
                            partition_aggregation = "winsorized_mean")

# View the corrected IV coefficient and its standard error
print(results)
```

`lpmec_multivariate` *Aggregated multivariate latent-predictor correction*

Description

Runs `lpmec_multivariate_onerun` over repeated split-half partitions and optional row bootstrap samples.

Usage

```
lpmec_multivariate(
  Y,
  observables,
  covariates = NULL,
  observables_groupings = NULL,
  make_observables_groupings = FALSE,
  n_boot = 32L,
  n_partition = 10L,
  partition_aggregation = "median",
  partition_aggregation_probs = c(0.01, 0.99),
  boot_basis = seq_along(Y),
  return_intermediaries = TRUE,
  estimation_method = "em",
  latent_estimation_fn = NULL,
  ordinal = FALSE,
 mcmc_control = list(backend = "pscl", n_samples_warmup = 500L, n_samples_mcmc = 1000L,
    batch_size = 512L, chain_method = "parallel", subsample_method = "full",
    anchor_parameter_id = NULL, n_thin_by = 1L, n_chains = 2L, outcome_prior =
    list(calibration = "data"), joint2_prior = list()),
  min_split_correlation = 0,
  conda_env = "lpmec",
  conda_env_required = FALSE
)
```

Arguments

`Y`	Numeric outcome vector.
`observables`	A list of matrices or data frames, one per latent predictor.
`covariates`	Optional matrix or data frame of observed covariates.
`observables_groupings`	Optional list of grouping vectors, one per latent predictor. Defaults to the column names of each observable matrix.
`make_observables_groupings`	Logical scalar or vector passed to `lpmec_onerun` for each latent predictor.
`n_boot`	Non-negative integer number of row-bootstrap iterations.
`n_partition`	Positive integer number of split-half partitions per original or bootstrap sample.
`partition_aggregation`	Aggregation strategy across partitions. See `lpmec`.
`partition_aggregation_probs`	Quantile probabilities for winsorized or trimmed partition aggregation.
`boot_basis`	Optional vector of indices or strata for row bootstrap.
`return_intermediaries`	Logical. If `TRUE`, returns per-run coefficient matrices.
`estimation_method`	Character scalar or vector passed to `lpmec_onerun` for each latent predictor.
`latent_estimation_fn`	Optional function or list of functions used when `estimation_method = "custom"`.
`ordinal`	Logical scalar or vector passed to `lpmec_onerun`.

mcmc_control List passed to lpmec_onerun.

min_split_correlation
Minimum allowed split-half correlation. The correction is defined only for positive componentwise correlations.

conda_env Character string naming the conda environment for MCMC methods.

conda_env_required
Logical indicating whether conda_env is required.

Value

A list of class lpmec_multivariate containing aggregated uncorrected OLS and corrected IV latent coefficients with bootstrap uncertainty summaries when n_boot >= 1.

---

lpmec_multivariate_onerun

Multivariate latent-predictor measurement-error correction

Description

Implements the split-indicator multivariate IV correction for several latent predictors. Each latent predictor is estimated from its own indicator matrix.

Usage

```
lpmec_multivariate_onerun(
  Y,
  observables,
  covariates = NULL,
  observables_groupings = NULL,
  make_observables_groupings = FALSE,
  estimation_method = "em",
  latent_estimation_fn = NULL,
  ordinal = FALSE,
 mcmc_control = list(backend = "pscl", n_samples_warmup = 500L, n_samples_mcmc = 1000L,
    batch_size = 512L, chain_method = "parallel", subsample_method = "full",
    anchor_parameter_id = NULL, n_thin_by = 1L, n_chains = 2L, outcome_prior =
    list(calibration = "data"), joint2_prior = list()),
  min_split_correlation = 0,
  conda_env = "lpmec",
  conda_env_required = FALSE
)
```

Arguments

Y Numeric outcome vector.

observables A list of matrices or data frames, one per latent predictor.

covariates Optional matrix or data frame of observed covariates.

observables_groupings
Optional list of grouping vectors, one per latent predictor. Defaults to the column names of each observable matrix.

`make_observables_groupings`
: Logical scalar or vector passed to `lpmec_onerun` for each latent predictor.

`estimation_method`
: Character scalar or vector passed to `lpmec_onerun` for each latent predictor.

`latent_estimation_fn`
: Optional function or list of functions used when `estimation_method = "custom"`.

`ordinal`
: Logical scalar or vector passed to `lpmec_onerun`.

`mcmc_control`
: List passed to `lpmec_onerun`.

`min_split_correlation`
: Minimum allowed split-half correlation. The correction is defined only for positive componentwise correlations.

`conda_env`
: Character string naming the conda environment for MCMC methods.

`conda_env_required`
: Logical indicating whether `conda_env` is required.

Value

A list of class `lpmec_multivariate_onerun` containing uncorrected OLS coefficients, uncorrected IV coefficients, corrected IV coefficients, split-half correlations, first-stage diagnostics, and latent score matrices.

`lpmec_onerun` *lpmec_onerun*

Description

Implements analysis for latent variable models with measurement error correction

Usage

```
lpmec_onerun(
  Y,
  observables,
  observables_groupings = colnames(observables),
  make_observables_groupings = FALSE,
  estimation_method = "em",
  latent_estimation_fn = NULL,
 mcmc_control = list(backend = "pscl", n_samples_warmup = 500L, n_samples_mcmc = 1000L,
   batch_size = 512L, chain_method = "parallel", subsample_method = "full", n_thin_by =
   1L, n_chains = 2L, outcome_prior = list(calibration = "data"), joint2_prior = list()),
  ordinal = FALSE,
  conda_env = "lpmec",
  conda_env_required = FALSE,
  partition = NULL,
  partition_id = NULL
)
```

Arguments

`Y` A vector of observed outcome variables

`observables` A matrix of observable indicators used to estimate the latent variable

`observables_groupings`
A vector specifying groupings for the observable indicators. Default is column names of observables.

`make_observables_groupings`
Logical. If TRUE, creates dummy variables for each level of the observable indicators. Default is FALSE.

`estimation_method`
Character specifying the estimation approach. Options include:

- "em" (default): Uses expectation-maximization via `emIRT` package. Supports both binary (via `emIRT::binIRT`) and ordinal (via `emIRT::ordIRT`) indicators.
- "pca": First principal component of observables.
- "averaging": Uses feature averaging.
- "mcmc": Markov Chain Monte Carlo estimation using either `pscl::ideal` (R backend) or `numpyro` (Python backend)
- "mcmc_joint": Joint Bayesian model that simultaneously estimates latent variables and outcome relationship using `numpyro`
- "mcmc_joint2": NumPyro mixed factor-analysis benchmark with binary indicators and continuous `Y` in one factor model
- "mcmc_overimputation": Two-stage MCMC approach with measurement error correction via over-imputation
- "custom": In this case, latent estimation performed using `latent_estimation_fn`.

`latent_estimation_fn`
Custom function for estimating latent trait from `observables` if `estimation_method="custom"` (optional). The function should accept a matrix of observables (rows are observations) and return a numeric vector of length equal to the number of observations.

`mcmc_control` A list indicating parameter specifications if MCMC used.

`backend` Character string indicating the MCMC engine to use. Valid options are `"pscl"` (default, uses the R-based `pscl::ideal` function) or `"numpyro"` (uses the Python numpyro package via reticulate).

`n_samples_warmup` Integer specifying the number of warm-up (burn-in) iterations before samples are collected. Default is `500`.

`n_samples_mcmc` Integer specifying the number of post-warmup MCMC iterations to retain. Default is `1000`.

`batch_size` Integer row subsample size used when `subsample_method = "batch"` with the NumPyro backend. Must be between `1` and `nrow(observables) - 1`. Default is `512`.

`subsample_method` Character string for NumPyro likelihood evaluation. Use `"full"` (default) for all rows or `"batch"` for experimental HMCECS row subsampling with `batch_size`.

`chain_method` Character string passed to numpyro specifying how to run multiple chains. Options: `"parallel"` (default), `"sequential"`, or `"vectorized"`.

`anchor_parameter_id` Optional 1-based observable-column index used by NumPyro MCMC backends to anchor item difficulty orientation. If omitted or `NULL`, automatic orientation is used.

n_thin_by Integer indicating the thinning factor for MCMC samples. Default is 1.

n_chains Integer specifying the number of parallel MCMC chains to run. Default is 2.

outcome_prior List controlling "mcmc_joint" outcome-model priors for the NumPyro backend. By default, calibration = "data" centers the intercept prior at mean(Y) and scales intercept, slope, and residual-sigma priors by sd(Y). Use calibration = "legacy" to restore the previous unit-scale priors, or provide numeric overrides for intercept_mean, intercept_sd, slope_mean, slope_sd, and sigma_sd. The optional scale_floor sets the minimum scale used for data-calibrated priors.

joint2_prior List controlling "mcmc_joint2" priors. Defaults are lambda_mean = 0, lambda_sd = 2, psi_shape = 0.0005, and psi_scale = 0.0005. For item loadings, lambda_mean and lambda_sd parameterize the raw loading scale before the positive softplus transform; all item loadings are positive by default.

ordinal Logical indicating whether the observable indicators are ordinal (TRUE) or binary (FALSE).

conda_env A character string specifying the name of the conda environment to use via reticulate. Default is "lpmec".

conda_env_required
A logical indicating whether the specified conda environment must be strictly used. If TRUE, an error is thrown if the environment is not found. Default is FALSE.

partition Optional fixed split-half partition. When supplied, must be a list with split1_names and split2_names entries naming observable groups. Default is NULL, which preserves the historical one-off random split behavior.

partition_id Optional identifier for partition; stored in the returned object for bootstrap diagnostics.

Details

This function implements a latent variable analysis with measurement error correction. It splits the observable indicators into two sets, estimates latent variables using each set, and then applies various correction methods including OLS correction and instrumental variable approaches.

Value

A list containing various estimates and statistics:

- Naive, IV, corrected IV, and corrected OLS estimates: ols_*, iv_*, corrected_iv_*, and corrected_ols_*. Split-specific estimates use suffixes _a and _b.
- var_est_split: Estimated split-half measurement-error variance.
- bayesian_ols_*_outer_normed and bayesian_ols_*_inner_normed: MCMC coefficient summaries when an MCMC method is used; otherwise NA.
- m_stage_1_erv and m_reduced_erv: Extreme robustness values for the first-stage and reduced-form regressions.
- outcome_prior_*: Resolved outcome-prior values used by NumPyro joint outcome models.
- mcmc_joint2_*: NumPyro "mcmc_joint2" diagnostics, including effective-sample-size percentages, maximum R-hat, divergent transitions, mean accept probability, and orientation diagnostics.

- `x_est1` and `x_est2`: Split-half latent variable estimates.

Standard Errors

The following single-run standard errors and t-statistics are currently returned as `NA` because their analytical derivation is not yet implemented:

- `corrected_iv_se`: Standard error for the corrected IV coefficient
- `corrected_ols_se`: Standard error for the corrected OLS coefficient
- `corrected_ols_tstat`: T-statistic for the corrected OLS coefficient
- `corrected_ols_coef_alt`: Alternative corrected OLS coefficient

For inference on these quantities, use the bootstrap approach via `lpmec`, which provides valid confidence intervals and standard errors through resampling.

Examples

```
# Generate some example data
set.seed(123)
Y <- rnorm(1000)
observables <- as.data.frame(matrix(sample(c(0,1), 1000*10, replace = TRUE), ncol = 10))

# Run the analysis
results <- lpmec_onerun(Y = Y,
                        observables = observables)

# View the corrected estimates
print(results)
```

`plot.lpmec` *Plot method for lpmec objects*

Description

Creates visualizations of LPMEC model results. Can plot either the latent variable estimates or the bootstrap distribution of coefficients.

Usage

```
## S3 method for class 'lpmec'
plot(x, type = "latent", ...)
```

Arguments

`x`	An object of class `lpmec` returned by `lpmec`.
`type`	Character string specifying the plot type. Either `"latent"` (default) for a scatter plot of split-half latent estimates, or `"coefficients"` for a density plot of bootstrap coefficient estimates.
`...`	Additional arguments passed to `plot` or `density`.

Value

No return value, called for side effects (creates a plot).

See Also

lpmec, summary.lpmec, print.lpmec

---

plot.lpmec_onerun *Plot method for lpmec_onerun objects*

Description

Creates a scatter plot comparing the two split-half latent variable estimates.

Usage

```
## S3 method for class 'lpmec_onerun'
plot(x, ...)
```

Arguments

x	An object of class lpmec_onerun returned by lpmec_onerun.
...	Additional arguments passed to plot.

Value

No return value, called for side effects (creates a plot).

See Also

lpmec_onerun, summary.lpmec_onerun, print.lpmec_onerun

---

print.lpmec *Print method for lpmec objects*

Description

Prints a concise summary of bootstrapped LPMEC model results.

Usage

```
## S3 method for class 'lpmec'
print(x, ...)
```

Arguments

x	An object of class lpmec returned by lpmec.
...	Additional arguments (currently unused).

Value

The input object x, returned invisibly.

See Also

lpmec, summary.lpmec, plot.lpmec

---

print.lpmec_multivariate *Print method for lpmec_multivariate objects*

Description

Print method for lpmec_multivariate objects

Usage

```
## S3 method for class 'lpmec_multivariate'
print(x, ...)
```

Arguments

x	An object of class lpmec_multivariate.
...	Additional arguments (currently unused).

Value

The input object x, returned invisibly.

---

print.lpmec_multivariate_onerun *Print method for lpmec_multivariate_onerun objects*

Description

Print method for lpmec_multivariate_onerun objects

Usage

```
## S3 method for class 'lpmec_multivariate_onerun'
print(x, ...)
```

Arguments

x	An object of class lpmec_multivariate_onerun.
...	Additional arguments (currently unused).

Value

The input object x, returned invisibly.

print.lpmec_onerun *Print method for lpmec_onerun objects*

Description

Prints a concise summary of single-run LPMEC model results.

Usage

```
## S3 method for class 'lpmec_onerun'
print(x, ...)
```

Arguments

x	An object of class lpmec_onerun returned by lpmec_onerun.
...	Additional arguments (currently unused).

Value

The input object x, returned invisibly.

See Also

lpmec_onerun, summary.lpmec_onerun, plot.lpmec_onerun

summary.lpmec *Summary method for lpmec objects*

Description

Provides a comprehensive summary of bootstrapped LPMEC model results including OLS, IV, corrected, and Bayesian coefficient estimates with confidence intervals.

Usage

```
## S3 method for class 'lpmec'
summary(object, ...)
```

Arguments

object	An object of class lpmec returned by lpmec.
...	Additional arguments (currently unused).

Value

A data frame containing coefficient estimates, standard errors, and confidence intervals, returned invisibly. The data frame has rows for OLS, IV, Corrected IV, Corrected OLS, and Bayesian OLS estimates.

See Also

lpmec, print.lpmec, plot.lpmec

summary.lpmec_multivariate *Summary method for lpmec_multivariate objects*

Description

Summary method for lpmec_multivariate objects

Usage

```
## S3 method for class 'lpmec_multivariate'
summary(object, ...)
```

Arguments

object	An object of class lpmec_multivariate.
...	Additional arguments (currently unused).

Value

A data frame of aggregated latent-predictor coefficient estimates.

summary.lpmec_multivariate_onerun *Summary method for lpmec_multivariate_onerun objects*

Description

Summary method for lpmec_multivariate_onerun objects

Usage

```
## S3 method for class 'lpmec_multivariate_onerun'
summary(object, ...)
```

Arguments

object	An object of class lpmec_multivariate_onerun.
...	Additional arguments (currently unused).

Value

A data frame of latent-predictor coefficient estimates.

`summary.lpmec_onerun` *Summary method for lpmec_onerun objects*

Description

Provides a summary of single-run LPMEC model results including OLS, IV, and corrected coefficient estimates.

Usage

```
## S3 method for class 'lpmec_onerun'
summary(object, ...)
```

Arguments

`object`	An object of class `lpmec_onerun` returned by `lpmec_onerun`.
`...`	Additional arguments (currently unused).

Value

A data frame containing coefficient estimates and standard errors, returned invisibly. The data frame has rows for OLS, IV, Corrected IV, and Corrected OLS estimates.

See Also

`lpmec_onerun`, `print.lpmec_onerun`, `plot.lpmec_onerun`

Index